CSAI 204 – Operating Systems

Laboratory Report – Lab 3

Task 3: Ping Pong

1. Objective

This experiment focused on implementing communication between two processes in xv6 using Unix pipes. The parent process transfers a byte to the child process, while the child acknowledges the message and returns the byte back to the parent. The program demonstrates synchronization and bidirectional IPC.

2. Background

A pipe is a mechanism used for exchanging data between related processes. In xv6, every pipe provides a read descriptor and a write descriptor. Since communication only moves in one direction per pipe, two separate pipes are needed to allow both the parent and child to exchange data.

3. Implementation

3.1 Creating the Pipes
The program initializes two arrays representing the communication channels. One pipe handles messages from the parent to the child, while the other is responsible for the reverse direction.

3.2 Child Process Logic
After calling fork(), the child process closes all unnecessary file descriptors. It waits to receive a byte from the parent process, prints a confirmation message containing its PID, and then sends the same byte back through the second pipe.

3.3 Parent Process Logic
The parent process transmits a single character to the child, pauses until the child completes execution, and finally reads the returned byte. Once the communication succeeds, the parent prints the pong message.

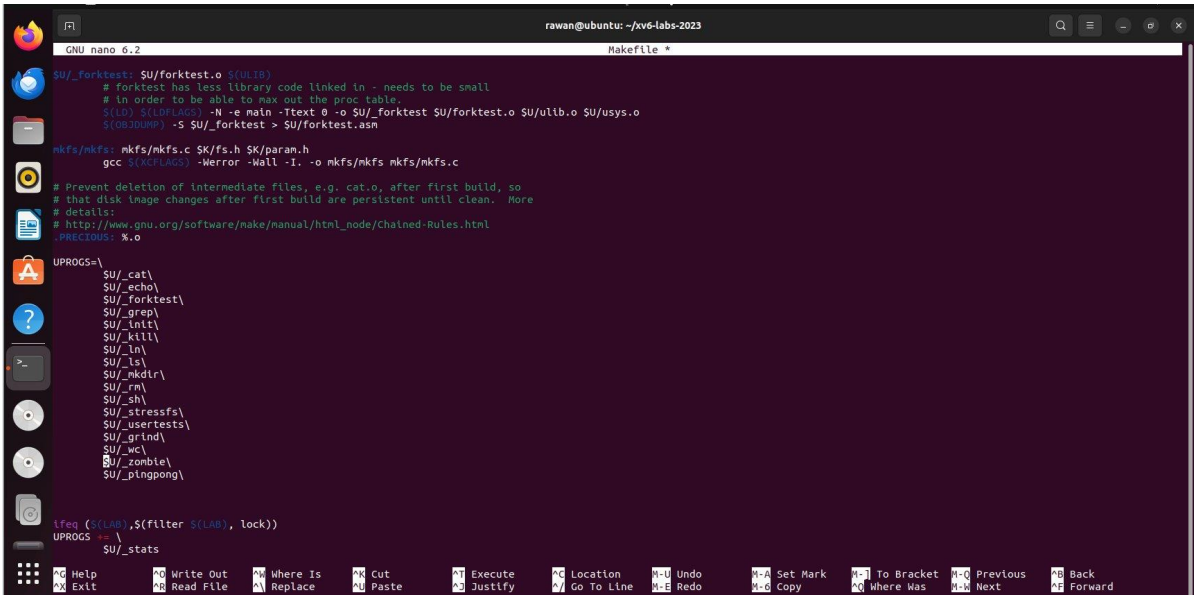
3.4 Updating the Makefile
The pingpong program was added to the xv6 build configuration by inserting _pingpong into the UPROGS section of the Makefile. This guarantees the executable is included during compilation.

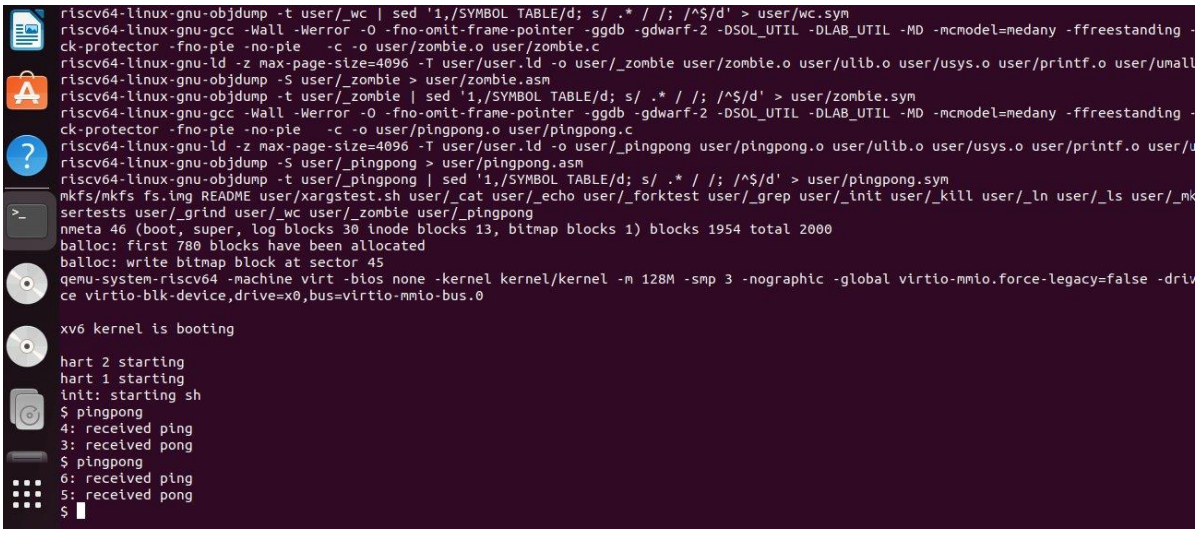
4. Results & Output

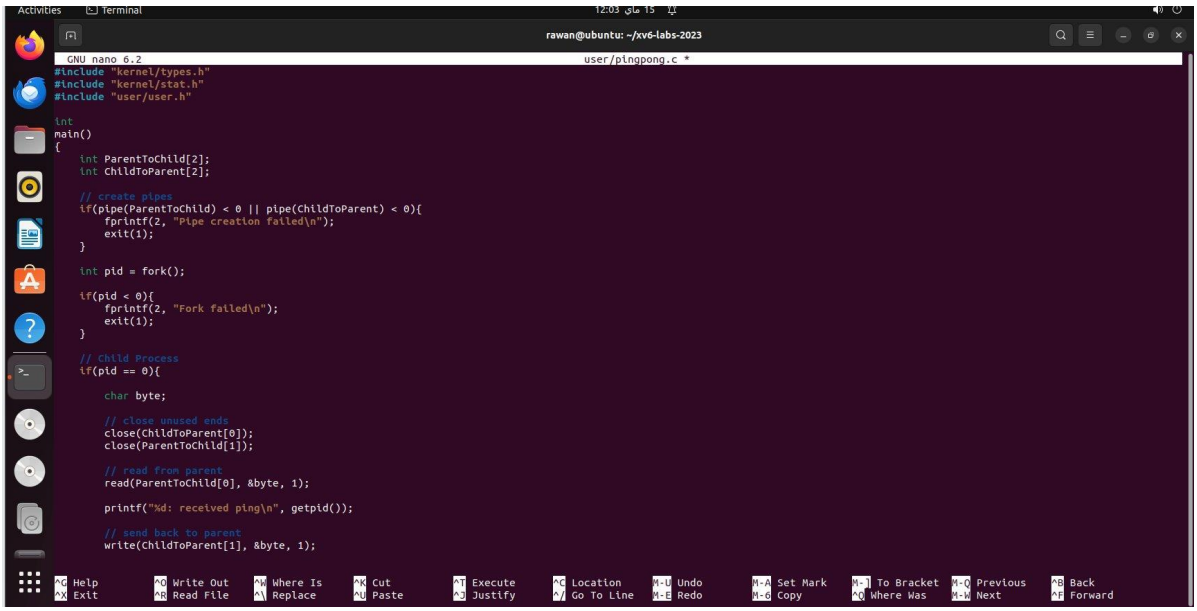
The application was compiled successfully and executed within the xv6 environment using QEMU. The terminal output verified that the parent and child exchanged messages correctly. The child displayed the ping notification first, followed by the parent receiving the pong response.

5. Screenshots

The following screenshots illustrate the implementation, Makefile configuration, and final execution results.


rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 Makefile *
$U/_forktest: $U/forktest.o $(ULIB)
forktest has less library code linked in - needs to be small
in order to be able to max out the proc table.
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/forktest.o $U/ulib.o $U/usys.o
$(OBJDUMP) -S $U/_forktest > $U/forktest.asm
mkfs/mkfs: mkfs/mkfs.c $K/fs.h $K/param.h
gcc $(XCFLAGS) -Werror -Wall -I. -o mkfs/mkfs mkfs/mkfs.c
Prevent deletion of intermediate files, e.g. cat.o, after first build, so
that disk image changes after first build are persistent until clean. More
details:
http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
.PRECIOUS: %.o
UPROGS=\
$U/_cat\
$U/_echo\
$U/_forktest\
$U/_grep\
$U/_init\
$U/_kill\
$U/_ln\
$U/_ls\
$U/_mkdir\
$U/_rm\
$U/_sh\
$U/_stressfs\
$U/_usertests\
$U/_grind\
$U/_wc\
$U/_zombie\
$U/_pingpong\
ifeq ($(LAB),$(filter $(LAB), lock))
UPROGS += \
$U/_stats
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous ^B Back
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next ^F Forward


riscv64-linux-gnu-objdump -t user/_wc | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/wc.sym
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -
ck-protector -fno-pie -no-pie -c -o user/zombie.o user/zombie.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_zombie user/zombie.o user/ulib.o user/usys.o user/printf.o user/umall
riscv64-linux-gnu-objdump -S user/_zombie > user/zombie.asm
riscv64-linux-gnu-objdump -t user/_zombie | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/zombie.sym
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -
ck-protector -fno-pie -no-pie -c -o user/pingpong.o user/pingpong.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_pingpong user/pingpong.o user/ulib.o user/usys.o user/printf.o user/u
riscv64-linux-gnu-objdump -S user/_pingpong > user/pingpong.asm
riscv64-linux-gnu-objdump -t user/_pingpong | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/pingpong.sym
mkfs/mkfs fs.img README user/xargstest.sh user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mk
sertests user/_grind user/_wc user/_zombie user/_pingpong
nmeta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 1954 total 2000
balloc: first 780 blocks have been allocated
balloc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -driv
ce virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ pingpong
4: received ping
3: received pong
$ pingpong
6: received ping
5: received pong
$


Activities Terminal 12:03 ماق 15
rawan@ubuntu: ~/xv6-labs-2023
GNU nano 6.2 user/pingpong.c *
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
int
main()
{
int ParentToChild[2];
int ChildToParent[2];
// create pipes
if(pipe(ParentToChild) < 0 || pipe(ChildToParent) < 0){
fprintf(2, "Pipe creation failed\n");
exit(1);
}
int pid = fork();
if(pid < 0){
fprintf(2, "Fork failed\n");
exit(1);
}
// Child Process
if(pid == 0){
char byte;
// close unused ends
close(ChildToParent[0]);
close(ParentToChild[1]);
// read from parent
read(ParentToChild[0], &byte, 1);
printf("%d: received ping\n", getpid());
// send back to parent
write(ChildToParent[1], &byte, 1);
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous ^B Back
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next ^F Forward

6. Conclusion

This task demonstrated how pipes can be used to implement process communication in xv6. The experiment highlighted the importance of managing file descriptors carefully and synchronizing parent and child processes correctly. The final output confirmed that the IPC mechanism worked as expected.